

FORMATION EMBARQUÉE

Module 03 — Arduino

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

Module 03 — Arduino : l'embarqué par la pratique

Arduino = une carte à microcontrôleur + un IDE + un écosystème de bibliothèques. C'est le meilleur terrain d'entraînement : chaque concept des modules 00-02 s'y touche du doigt. Le langage est du **C++** avec deux fonctions imposées : `setup()` et `loop()` .

1. Le matériel

1.1 Cartes courantes

Carte	Micro	Flash/RAM	Tension	Points forts
Uno / Nano	ATmega328P (8 bits, 16 MHz)	32 Ko / 2 Ko	5 V	La référence pour apprendre
Mega 2560	ATmega2560	256 Ko / 8 Ko	5 V	Beaucoup de broches
ESP32	Xtensa 32 bits, 240 MHz	4 Mo / 520 Ko	3,3 V	Wi-Fi + Bluetooth intégrés
ESP8266	32 bits, 80 MHz	4 Mo / 80 Ko	3,3 V	Wi-Fi pas cher
RP2040 (Pico)	2 cœurs ARM M0+	2 Mo / 264 Ko	3,3 V	PIO (E/S programmables)

Commence sur **Uno ou Nano** (robuste, 5 V, documentation infinie), passe à l'**ESP32** dès que tu veux du réseau.

1.2 Broches de l'Uno

- **D0–D13** : numériques (entrée/sortie). D0/D1 = UART (éviter). Les broches marquées  (3, 5, 6, 9, 10, 11) font du **PWM**.
- **A0–A5** : entrées analogiques (ADC 10 bits). A4/A5 = I2C (SDA/SCL).
- **D10–D13** : SPI (CS, MOSI, MISO, SCK).
- Alimentation : 5 V, 3,3 V, GND, Vin (7–12 V).
- ⚠ Max ~20 mA par broche : une LED exige sa **résistance série** (220 Ω) ; un moteur exige un **transistor ou driver** — jamais en direct.

1.3 Outils

- **IDE Arduino 2.x** (simple) ou **VS Code + PlatformIO** (pro : gestion de bibliothèques par projet, debug, multi-cartes). Passe à PlatformIO dès que tu es à l'aise.
- Simulateurs sans matériel : **Wokwi** (<https://wokwi.com>, gratuit, dans le navigateur) et Tinkercad Circuits.

2. Structure d'un sketch

```
// Constantes de câblage en tête de fichier
const uint8_t BROCHE_LED = 13;

void setup() {                                // exécuté UNE fois au démarrage
    pinMode(BROCHE_LED, OUTPUT);
    Serial.begin(115200);                     // ouvre le port série vers le PC
}

void loop() {                                 // exécuté en boucle, indéfiniment
    digitalWrite(BROCHE_LED, HIGH);
    delay(500);                               // bloque 500 ms (on fera mieux, §5)
    digitalWrite(BROCHE_LED, LOW);
    delay(500);
}
```

En coulisses, l'environnement fournit un `main()` qui appelle `setup()` puis `loop()` en boucle infinie — exactement le squelette bare-metal du module 01.

3. Les E/S fondamentales

3.1 Numérique

```
pinMode(7, INPUT_PULLUP);                    // bouton entre broche et GND
bool appuye = (digitalRead(7) == LOW);       // pull-up → LOW = appuyé (logique inversée)

pinMode(13, OUTPUT);
digitalWrite(13, HIGH);
```

3.2 Analogique

```
int brut = analogRead(A0);                   // 0..1023 pour 0..5 V
float volts = brut * (5.0 / 1023.0);
int pct = map(brut, 0, 1023, 0, 100);        // règle de trois intégrée

analogWrite(9, 128);                         // PWM ~50 % (0..255) – pas un vrai DAC !
```

3.3 Port série (ton meilleur ami pour déboguer)

```
Serial.begin(115200);
Serial.print("temp = ");
Serial.println(temperature);

if (Serial.available()) {           // des octets sont arrivés du PC
    char c = Serial.read();
    if (c == '1') digitalWrite(13, HIGH);
}
```

Le **Moniteur série** de l'IDE affiche ces messages ; le **Traceur série** trace les valeurs en courbes.

4. Capteurs et actionneurs classiques

```
// ---- Potentiomètre → luminosité LED
int pot = analogRead(A0);
analogWrite(9, pot / 4);           // 0..1023 → 0..255

// ---- Capteur ultrason HC-SR04 (distance)
digitalWrite(TRIG, HIGH); delayMicroseconds(10); digitalWrite(TRIG, LOW);
long duree = pulseIn(ECHO, HIGH); // µs aller-retour
float cm = duree / 58.0;

// ---- Servo-moteur
#include <Servo.h>
Servo s;
s.attach(9);
s.write(90);                       // angle 0..180°

// ---- Température/humidité DHT22 (bibliothèque "DHT sensor library")
#include <DHT.h>
DHT dht(2, DHT22);
dht.begin();
float t = dht.readTemperature();

// ---- I2C : écran OLED SSD1306 (bibliothèque Adafruit SSD1306)
// ---- SPI : carte SD (bibliothèque SD)
```

Installer une bibliothèque : IDE → Croquis → Inclure une bibliothèque → Gérer les bibliothèques. Avec PlatformIO : `lib_deps` dans `platformio.ini`.

5. LE tournant : programmer sans `delay()`

`delay()` fige tout : impossible de surveiller un bouton pendant qu'une LED clignote. La solution : horodater avec `millis()` (millisecondes depuis le démarrage) et tester le temps écoulé.

```

const uint32_t PERIODE = 500;
uint32_t derniere_bascule = 0;

void loop() {
    uint32_t maintenant = millis();

    if (maintenant - derniere_bascule >= PERIODE) { // soustraction : robuste
        derniere_bascule = maintenant;           // au débordement 49 jours
        digitalWrite(13, !digitalRead(13));
    }

    // ...la boucle reste libre pour lire boutons, capteurs, série...
}

```

Ce motif (« blink without delay ») + la machine d'états du module 01 = tu peux gérer *plusieurs* activités simultanées sur un seul micro sans OS. C'est la compétence Arduino qui sépare débutant et intermédiaire.

6. Interruptions

```

volatile uint32_t compte_tours = 0; // volatile : partagé avec l'ISR !

void isr_capteur() { // ISR : courte, pas de Serial, pas de delay
    compte_tours++;
}

void setup() {
    pinMode(2, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(2), isr_capteur, FALLING);
}

void loop() {
    noInterrupts(); // section critique : lecture atomique
    uint32_t copie = compte_tours;
    interrupts();
    // ...utiliser copie...
}

```

Sur Uno, seules D2 et D3 ont des interruptions externes ; sur ESP32, presque toutes les broches.

7. Projet fil rouge : station météo connectée

Monte en compétence par étapes — chaque étape est un projet fonctionnel :

1. **Blink** puis blink sans `delay()` . (GPIO, millis)
2. **Bouton anti-rebond** qui change le mode de clignotement. (entrées, FSM)
3. **Lecture DHT22** affichée sur le moniteur série toutes les 2 s. (capteur, bibliothèque, timing non bloquant)
4. **Écran OLED I2C** qui affiche température + humidité. (I2C)

5. **Enregistrement sur carte SD** en CSV avec horodatage RTC. (*SPI, fichiers*)
6. **Version ESP32** : envoi des mesures en Wi-Fi (HTTP ou MQTT vers un broker Mosquitto). (*réseau, JSON*)
7. **Alerte** : seuil réglable par potentiomètre, buzzer + LED rouge, hystérésis pour éviter le clignotement autour du seuil. (*ADC, logique*)

À la fin tu auras utilisé : GPIO, ADC, PWM, I2C, SPI, UART, interruptions, FSM, réseau — c'est-à-dire *tout* le module 00 en vrai.

8. Bonnes pratiques et pièges Arduino

1. **int fait 16 bits sur Uno** : `millis()` retourne un `uint32_t`, stocke-le dans un `uint32_t`, pas un `int`.
 2. La RAM fait **2 Ko** : `Serial.println(F("texte"))` garde la chaîne en flash au lieu de la RAM.
 3. `String` (la classe Arduino) fragmente le tas → sur Uno, préférer les tableaux de `char`.
 4. Débranche D0/D1 pendant le téléversement si tu y as branché quelque chose.
 5. Alimente les moteurs/servos par une **alimentation séparée** (masses communes !) — les pics de courant redémarrent la carte.
 6. Structure ton code en fonctions/classes dès 50 lignes ; un sketch n'est pas obligé d'être un plat de spaghettis.
 7. Mesure la place : l'IDE affiche l'usage flash/RAM à chaque compilation.
-

9. Après Arduino : STM32 et le monde pro

Quand Arduino te semblera petit : - **STM32 "Blue Pill" / Nucleo + STM32CubeIDE** : tu configures les horloges et périphériques toi-même (HAL en C), tu débogues avec ST-Link. C'est l'environnement des offres d'emploi « firmware ». - Le chemin : refaire la station météo sur STM32 avec la HAL, puis avec **FreeRTOS** (module 06), puis en accès registres direct. - Tout cela est détaillé pas à pas dans le **Module 10 — STM32**.

Exercices

1. Chenillard de 4 LED, vitesse réglée par potentiomètre, sans `delay()`.
2. Compteur de passages : capteur ultrason + interruption timer, affichage OLED, remise à zéro par bouton (anti-rebond).
3. Thermostat : DHT22 + relais (simulé par LED) + hystérésis de 0,5 °C + consigne réglable, le tout en machine d'états.
4. Sur Wokwi si pas de matériel : reproduis les exercices 1 à 3.

→ Suite : **Module 04 — VHDL & FPGA** : on quitte le logiciel pour décrire du matériel.